Automated planning for Todoist

Matěj Kubík - 514143

June 2026

1 Project goal

The goal of this project is to bring automated planning to Todoist by continuously transforming a changing task list into a feasible and high-quality schedule. The system considers priority, estimated duration, and deadline as the main decision factors, then assigns flexible tasks to specific time slots inside a user-defined productivity window. Whenever tasks are created, updated, or completed, the planner recomputes the global schedule so that the result reflects the current state of the workspace rather than a stale snapshot. Fixed tasks, marked with the `fixed` label, are treated as immutable constraints, while flexible tasks are redistributed whenever a better allocation becomes available.

2 Description of the algorithm and theory, similar projects

2.1 Large Neighborhood Search (LNS)

Large Neighborhood Search is a metaheuristic designed for combinatorial optimization problems in which a complete solution can be improved by repeatedly disrupting and rebuilding parts of it. Instead of making only small local moves, LNS removes a selected subset of elements from the current solution, temporarily relaxing part of the search space, and then reconstructs the solution in a different way. This destroy-and-repair cycle makes it possible to escape local optima that would trap more conservative local search methods. The fundamental concepts were introduced by Shaw [1], and comprehensive surveys have expanded the methodology [2]. In the context of task planning, the approach is especially useful because moving a few tasks can change the feasibility and quality of the entire schedule.

2.2 Adaptive Large Neighborhood Search (ALNS)

Adaptive Large Neighborhood Search extends LNS by maintaining multiple destroy and repair operators and adapting their usage according to observed

performance [3]. The algorithm does not assume that one heuristic is universally best; instead, it learns which operators are more effective for the current instance and gradually prefers them during the search. In this project, operator selection is implemented through a roulette-wheel style mechanism, which increases the probability of choosing operators that recently produced better solutions. This makes ALNS well suited to planning problems with heterogeneous task distributions, because different schedules may benefit from different removal and reinsertion strategies.

2.3 Problem Formulation

The scheduling task can be viewed as an optimization problem whose decision variables are the start times of flexible tasks. The solution must satisfy several constraints at the same time: fixed tasks may not be moved, scheduled tasks must not overlap, all placements must stay within the available productivity window, and tasks should ideally finish before their deadlines. The objective function rewards schedules that place higher-priority tasks well, preserve buffer before deadlines, and avoid missed deadlines. This formulation captures the practical trade-off between urgency, duration, and temporal feasibility that appears in everyday task planning.

2.4 Similar Projects and Related Work

Commercial systems such as Motion [9] already demonstrate that automated planning can improve daily task management, but they are closed products, optimized for their own ecosystem, and not designed for experimentation with explicit destroy and repair operators. Their strength is convenience, while their limitation for this project is lack of transparency and lack of control over the underlying optimization strategy. Academic proposals such as ScheduleME [4] focus on student task management and intelligent scheduling, but they are typically oriented toward a mobile assistant scenario rather than a webhook-driven Todoist workflow. Similarly, Smart Scheduler [5] integrates tasks from multiple platforms and prioritizes them automatically, yet it targets broad task aggregation rather than continuous re-optimization of a single evolving Todoist workspace. These systems are relevant because they show the broader research direction, but they do not match the exact combination of incremental updates, fixed constraints, and operator-level evaluation used here.

3 Description of the implementation

3.1 Why LNS and ALNS for this problem?

The choice of ALNS follows naturally from the structure of the problem. The objective is also multi-dimensional: it rewards priority handling, deadline awareness, and feasible placement simultaneously. ALNS is a good fit because it can

combine several heuristics, let them compete on the same instance, and gradually emphasize the operators that work best for the current task mix. In practice, this gives the scheduler enough flexibility to handle dense task sets, fragmented calendars, and a mixture of long and short tasks without hard-coding one fixed strategy.

3.2 Core Components

The implementation is organized around a base planner abstraction, a dedicated ALNS driver, and a set of destroy and repair operators that define the search behavior. The planner layer in the `planners` package coordinates optimization, while the `alns_components` subpackage contains the pieces that modify and evaluate candidate schedules. This separation keeps the search logic independent from the Todoist integration and makes it easier to evaluate different heuristic combinations in tests.

3.2.1 Destroy Operators

The destroy operators focus on removing parts of the current schedule so that the repair stage can explore alternative placements. Random destroy (RD) removes tasks without strong bias and therefore serves as a simple exploration mechanism. Its main advantage is speed and diversity; its main weakness is that it may remove valuable tasks even when a more selective choice would be preferable. Lowest objective contribution destroy (LOCD) removes tasks that currently contribute least to the objective. This makes it a more directed heuristic and often a useful baseline, but the same selectivity can also reduce exploration and increase runtime because each candidate task must be evaluated carefully. Short-task-clusters destroy (STCD) targets groups of short tasks that fragment the schedule. It is useful when long tasks fail primarily because free blocks are broken into too many small intervals, although it provides less benefit when the main bottleneck lies elsewhere. Random-duration-destroy (RDD) introduces randomness while still taking task duration into account, which often produces a better balance between exploration and problem awareness than a purely uniform random strategy.

3.2.2 Repair Operators

The repair stage reinserts removed tasks into the schedule and therefore has a strong influence on the final solution quality. Regret repair (RR) inserts tasks according to regret, meaning that it prioritizes tasks whose alternative placements would be much worse [6, 7]. This makes the operator effective for tasks that are difficult to place, especially when the schedule is already tight, but the extra evaluation cost makes it slower than simpler methods. Simple-heuristic repair (SHR) instead reinserts tasks greedily to first available slot in priority + time-before-deadline order, which keeps it fast and easy to reason about. The trade-off is that it may accept locally reasonable placements that

later block better global solutions. For that reason, RR is usually the stronger method when solution quality matters more than raw speed.

3.3 Objective Function

The objective function aggregates the contribution of all scheduled tasks. Conceptually, the planner maximizes a weighted sum of individual task scores, where the score depends on priority, deadline satisfaction, and whether the task was successfully scheduled before its deadline. Tasks without deadlines receive a reduced positive reward, tasks that miss their deadlines receive a strong penalty, and tasks that finish well before the deadline receive a higher score than tasks placed only narrowly in time. In the implementation, the reward grows exponentially with priority, so higher-priority tasks dominate lower-priority ones when the schedule is constrained. If we denote the contribution of task t by $c(t)$, then the planner seeks a schedule with a large total reward, while failed placements are mapped to a negative value to discourage infeasible or late allocations. This design makes the objective sensitive both to correctness and to practical usefulness, because a schedule is not considered good merely because it fits, but because it fits the right tasks in the right places.

$$\max\left(\sum_{t\in T} c(s)\right)$$

Where:

$$c(s) = \begin{cases} B_0^{P_t} & \text{if } t \text{ has no deadline} \\ F \cdot B^{P_t} & \text{if failed to schedule } t \text{ before its deadline} \\ (1+\beta)\cdot B^{P_t} & \text{otherwise} \end{cases}$$

Task properties

- P_t = task priority
- D_t = task deadline
- E_t = scheduled task end

Constants

- B = priority base
- B_0 = priority base for tasks without deadline
- F = failed penalty multiplier
- $e^*(t)$ = best possible end time (furthest from deadline)

Per-task contribution:
where

$$\beta = \frac{\text{scheduled distance from deadline}}{\text{best possible distance from deadline}}$$

3.4 Architecture Overview

The implementation follows a layered structure. The base planner defines a common interface, the ALNS planner implements the optimization workflow, and the problem state stores the current schedule together with the information needed by the search operators. State initializers construct the first feasible solution, after which the destroy and repair operators iteratively modify it. This division of responsibilities keeps the code understandable and allows the evaluation suite to compare operator combinations without changing the surrounding scheduling infrastructure.

4 Installation and startup instructions

4.1 Prerequisites

The project requires Python 3.13 or newer and uses PDM as the primary dependency manager. Full Todoist integration additionally depends on `cloudflared` and two Todoist Pro accounts, because task durations and webhook-based synchronization are only available in that configuration.

4.2 Setup

1. Extract the zip file containing the source code and navigate to the project directory.
2. Copy `.env_example` to `.env` and fill in the required values:

   ```
   cp .env_example .env
   ```

3. Install dependencies:

   ```
   pdm install
   ```

   This installs all required packages including: FastAPI, ALNS, Todoist API client, MLflow, and matplotlib.

4.3 Testing and Evaluation

The operator comparison experiment is launched with `pdm run test_comp`; it executes the predefined task seeds, compares multiple destroy and repair combinations, and writes the resulting tables and plots. For experiment tracking, `pdm`

`run mlflow` starts the MLflow user interface on port 8080 so that the search behavior and generated metrics can be inspected interactively.

Generated outputs: `best_objective_plot.png` — convergence plot comparing the tested methods; `best_objective.csv` — CSV with results for all methods across all test seeds; and the `test_schedules/` folder, which contains `comparisons_summary_bar.png` (bar plot comparing best objectives per model) plus schedule visualizations for each model/seed combination.

4.4 Running the application

For a simple demonstration, the planners can be used directly from Python without connecting to a live Todoist account. Full integration requires webhook configuration and a separate Todoist setup, which is described in detail in the README because it depends on external services and two Pro accounts. In practice, the demo mode is sufficient for testing the optimization logic, while the full integration is the path for real-world deployment.

5 Running examples (images, listings)

The evaluation compares six operator combinations. The first configuration uses LOCD together with RR and serves as a deterministic baseline. The second configuration adds STCD to that baseline to see whether clustering short tasks creates more useful blocks for long tasks. The third configuration uses RDD with RR and therefore emphasizes stochastic exploration. The fourth combines RDD with LOCD, which mixes randomness with a heuristic focus on low-value placements. The fifth uses RDD and STCD together, while the sixth combines all three destroy operators with RR and acts as the most flexible variant in the comparison.

The evaluation was carried out on predefined task sets stored in `task_seeds/`. Each seed contains a mixture of fixed and flexible tasks, multiple priority levels, randomized durations, fragmented free intervals, and deadlines distributed across the planning horizon. The generated outputs are summarized in the bar chart below, while the convergence plot shows how the best objective value evolved during the search.

The raw comparison data is stored in `test_comparisons.csv`, and the individual schedules generated during the tests are available in `test_schedules/`.

6 Description and results of the evaluation

The first part of the evaluation compares two representative planners on the same task instance. The baseline heuristic, based on LOCD + RR, produces a schedule that is feasible but noticeably looser, with more idle gaps and less effective use of the second half of the planning horizon. By contrast, RDD + RR yields a denser schedule and schedules more tasks in total. Although RDD + RR places some priority-1 tasks later than LOCD + RR, this trade-off is

Figure 1: Summary comparison of configurations

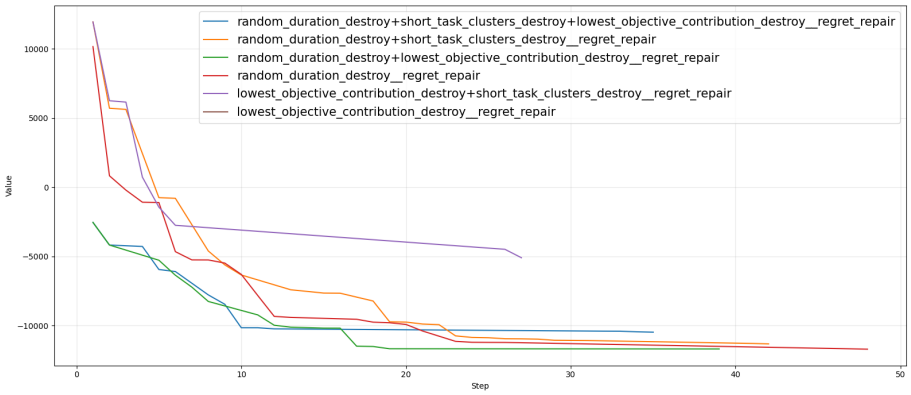

Figure 2: Objective convergence over iterations

beneficial because it leaves only two priority-4 tasks and one priority-3 task unscheduled, while LOCD + RR leaves one priority-2 task, one priority-4 task, and three priority-3 tasks unscheduled. In other words, the random destroy strategy uses exploration to keep more important work in the schedule, and the resulting layout is tighter, especially in the later days, which also leaves more contiguous free space for newly arriving tasks.

The convergence plot shows how the different operator combinations evolve during the search. The configuration LOCD + STCD + RR was expected to converge quickly in the first iterations because it combines two heuristics, but in practice it performed worse than the methods that include randomness even at the beginning of the search. The configuration RDD + STCD + RR started slightly less strongly, yet it became clearly better in later iterations because the random destroy component preserved exploration. The configuration RDD + STCD + LOCD + RR was expected to be the strongest variant because

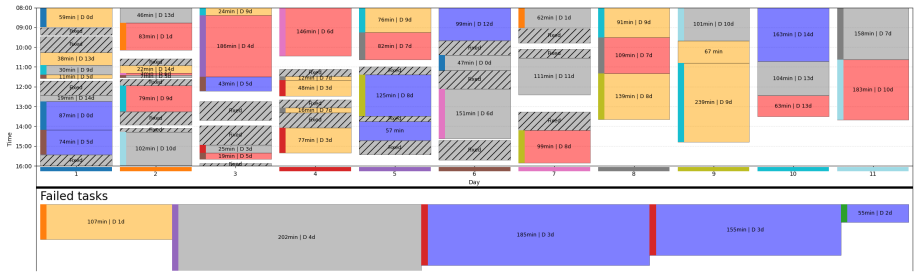

Figure 3: Representative schedule produced by the heuristic planner

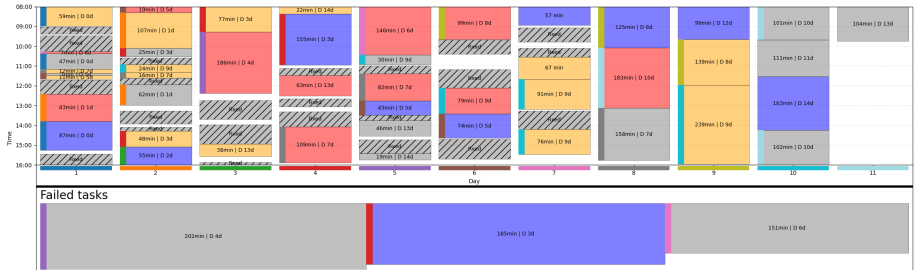

Figure 4: Representative schedule produced by the RD planner

it combines all destroy heuristics, but in total it achieved a worse score than the methods without short-task-cluster destruction. The configuration RDD + LOCD + RR converged faster than RDD + RR in the first iterations, but the pure random version became slightly better after more iterations because it continued to explore alternative placements more aggressively.

Overall, the STCD operator improved mainly the methods that did not already contain randomness. This suggests that random destroy already handled the formation of short-task clusters well enough on its own, so the additional cluster-based destruction brought only limited extra value. The most important factor across all comparisons was randomness, which appears necessary for escaping local optima and improving the schedule over time. At the same time, LOCD still contributed to faster initial convergence, which means it is useful as a heuristic component when the goal is to get a reasonable schedule quickly.

The following bar plot confirms the same pattern. The best average improvement over the baseline was achieved by RDD + RR with 378.64, followed by RDD + LOCD + RR with 364.98, RDD + STCD + RR with 358.58, and RDD + STCD + LOCD + RR with 351.37. The best heuristic-only combination, LOCD + STCD + RR, reached only 142.12, while the baseline LOCD + RR is the reference point with 0.0. These numbers correlate with the schedule visualizations above: methods with randomness produce tighter schedules and leave fewer important tasks unscheduled, whereas the more deterministic

heuristic approach converges faster at first but is less effective in the long run.

References

[1] Shaw, P. (1998). *Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems*. In: Principles and Practice of Constraint Programming. Springer, pp. 417–431.

[2] Pisinger, D., & Ropke, S. (2010). *Large Neighborhood Search*. In: Handbook of Metaheuristics. Springer, pp. 399–419.

[3] Ropke, S., & Pisinger, D. (2006). *An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows*. Transportation Science, 40(4), 455–472.

[4] ScheduleME. (n.d.). *ScheduleME: Smart Digital Personal Assistant for Automatic Priority Based Task Scheduling and Time Management*.

[5] Smart Scheduler. (n.d.). *Smart Scheduler: A Unified Intelligent System for Automated Task Management and Prioritization*.

[6] Potvin, J. Y., & Rousseau, J. M. (1993). *A Parallel Route Building Algorithm for the Vehicle Routing and Scheduling Problem with Time Windows*. European Journal of Operational Research, 66(3), 331–340.

[7] Diana, M., & Dessouky, M. (2004). *A New Regret Insertion Heuristic for Solving Large-Scale Dial-a-Ride Problems*. Transportation Research Part B: Methodological, 38(6), 539–557.

[8] Dueck, G. (1993). *New Optimization Heuristics: The Great Deluge Algorithm and the Record-to-Record Travel*. Journal of Computational Physics, 104(1), 86–92.

[9] Motion AI. (2024). *Motion: AI Calendar and Scheduling Platform*. Retrieved from https://www.usemotion.com/

[10] Todoist. (2024). *Todoist Developer Documentation: REST API and Webhooks*. Retrieved from https://developer.todoist.com/